

A Simulation-Assisted Framework for the Optimal Deployment of Robotic Operations with Static Contacts

Abstract—The deployment of robotic applications starts from a preliminary design stage, performed offline with the help of engineering tools, and ends with the physical assembly of the unit. This paper addresses the optimization of robotic work-cells for a specific class of tasks that involve static contacts, and is engineered for the *pre-deployment* phase. Nowadays, these tasks are often designed through experience-driven approaches, which can lead to sub-optimal results. We propose instead a bi-level optimization framework that models the overall design process as an iterative search towards the best feasible solution. The method jointly optimizes key design parameters, such as the work-cell layout and the robot configurations, while explicitly integrating the resolution of the task redundancy to handle the axial symmetry that usually characterizes such applications. The presented procedure is validated in simulation with two different use-cases, and several state-of-the-art algorithms are employed as benchmarks to validate our proposal.

I. INTRODUCTION

Robotic contact-based operations are a family of tasks that involve the application of a generalized force, usually referred to as *wrench*, at predefined target locations [1]. Some of these operations, such as screwing, unfastening, drilling, and many others, share the peculiarity that the motion range of the manipulator during the active execution phase is so restrained, that they can be considered as *static*.

Despite work-cells featured with robotic manipulators usually present higher flexibility if compared to hard-automation setups [2], their advantages can be completely lost if the working unit has not been carefully designed in the so-called *pre-deployment* phase, *i.e.*, before the physical unit is setup. In the literature, the optimization of robotic applications at this stage is mainly performed relying solely on the engineer’s expertise, according to *trial-and-error* strategies [3], that inevitably risk of jeopardizing the next phases. We believe instead that an adequate optimization strategy is of paramount importance for the overall application to be maintained in the long period, resulting in reduced efforts during the *sim-to-real* deployment.

With the aim of overcoming the limitations presented above, we propose the nested optimization pipeline schematized in Fig. 1. We formulate the problem according to the *leader-follower* paradigm, in order to manage the natural hierarchy between the variables involved: the overall layout of the work-cell is optimized at the upper level with the help of *Bayesian Optimization*, while the feasible robot configurations at each target location are determined at the lower level by means of a *normalizing flow*. More specifically, after a candidate layout is suggested by the leader, each follower optimizer computes the corresponding robot configurations for the target locations. The complete information is then

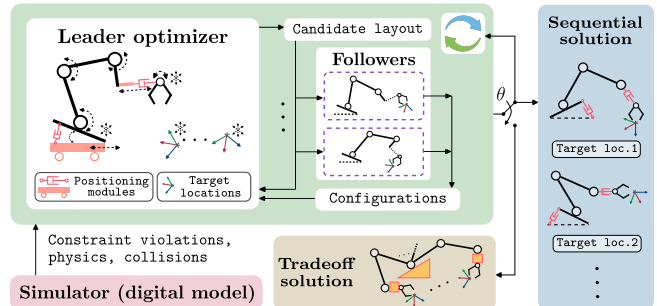


Fig. 1. Scheme of the implemented framework. In case of a tradeoff, the number of followers equals the one of target locations; vice versa, only one follower at a time is considered, and the framework is repeatedly executed for all the target locations.

fed back to the leader, which refines its proposal in the next iteration. A physics-based simulator is further integrated to accelerate computations and to check constraints that cannot be handled analytically. In addition, a binary selector (θ in Fig. 1) determines the type of optimization to be performed, allowing to find either a tradeoff solution (shared orange modules), or an optimal result for each one of the target locations. This is particularly useful in case of auxiliary positioning modules, such as external axis or mobile bases, that enable transitions between different layouts.

The method is meant to be applicable without restrictions on the selected manipulator or the auxiliary instances, such as tool, target locations or positioning modules. To the best of our knowledge, the definition of a simulation-assisted procedure allowing to orchestrate so many design variables at the same time has not been proposed yet in the literature.

II. STATE OF THE ART

The layout optimization problem has been extensively studied, as it directly impacts the task feasibility and efficiency. Early approaches [4], [5] target the spatial organization of the resources, relying mainly on heuristics, such as matrix and graphical representations. With the aim of increasing productivity and motion range, more recent works focus on the cycle time [6], [7], while [8] shows an optimization strategy to improve manipulability.

Selecting a reasonable organization of the elements in the work-cell is of pivotal importance also for contact-based operations, where reducing the actuators effort is usually the main element of concern [9]. In [10], a strategy concerning force maximization capabilities is detailed, while the framework shown in [11] allows to compensate for errors arising during machining tasks. Instead, [12] leverages hybrid

genetic algorithms to find the optimal placement of the robot base and the joints, minimizing the risk of singularities.

The axial symmetry of these tasks with respect to the wrench direction introduces the problem of *task redundancy*, that makes the resolution of the inverse kinematics (IK) more troublesome. In [13], the additional degree of freedom is exploited to minimize the actuators effort by implementing the extended Jacobian method [14]. Besides this algorithm, redundancy can be solved leveraging many different approaches [15], most of which are *local* and require a great deal of iterations to cover the search space. In addition to the sensitivity of these methods to the initial conditions, integrating complex constraints may lead solvers to inaccurate solutions or complete failure.

In this sense, the advent of industry 5.0 has fostered the adoption of computationally-efficient simulators [16], resulting in a renovated interest in *meta-heuristic* optimization [17]. The relevance of simulators can be appreciated not only for the implementation of offline *digital models* [18], but also when it comes to solving the IK of a robot [19], [20], accounting for complex collision constraints and leveraging *global* resolution schemes, whose adoption requires more computational resources than traditional *local* schemes.

With respect to the approaches discussed above, the main contributions of our methodology are the following:

- i) We propose a simulation-assisted pipeline for the offline design of robotized work-cells, characterized by a general applicability to a broad class of contact-based operations. Unlike previous methods [6], [12], our framework allows to jointly optimize all key design parameters, including robot-relevant variables and pose, target locations and tool attachment with respect to the end-effector.
- ii) By explicitly modeling task redundancy, our method allows to handle the complexity introduced by axial-symmetric tasks, relying on an efficient exploration of the feasible configuration space. Additionally, the formulation is not restricted to a specific kinematics, and can be applied to intrinsically redundant robots (e.g. mobile manipulators).

III. FORMALIZATION AND PROBLEM STATEMENT

Fig. 2 depicts the most general scenario for the tasks we target. For consistency, we introduce the notation $\mathbf{A}_k^i$ to reference the homogeneous transformation of a frame $\{\mathbf{h}_k\}$ with respect to $\{\mathbf{h}_i\}$. In addition, we denote as $\mathbf{x}_k^i = [\mathbf{t}_k^i, \phi_k^i]^\top \in \mathbb{R}^6$ the vector of the Cartesian coordinates encoded in $\mathbf{A}_k^i$ ($\mathbf{t}_k^i = [x_k^i, y_k^i, z_k^i]^\top$, while $\phi_k^i = [\theta_{x,k}^i, \theta_{y,k}^i, \theta_{z,k}^i]^\top$ is the minimal representation of the orientation in terms of Euler angles). The tasks of interest are featured with N_T locations where a certain action has to be executed by a robot (whose base is defined by $\{\mathbf{h}_b\}$, while $\{\mathbf{h}_{ee}\}$ defines the pose of the flange), characterized by ν degrees of freedom (DOFs). The robot is equipped with a suitable tool featured by $\{\mathbf{h}_t\}$ at its tip, and a specific generalized wrench $\mathbf{w}_j = [F_{x,j}, F_{y,j}, F_{z,j}, M_{x,j}, M_{y,j}, M_{z,j}]^\top \in \mathbb{R}^6$ must be guaranteed by the robot at each target location j . The static joint torques $\tau_j \in \mathbb{R}^\nu$ can be computed as

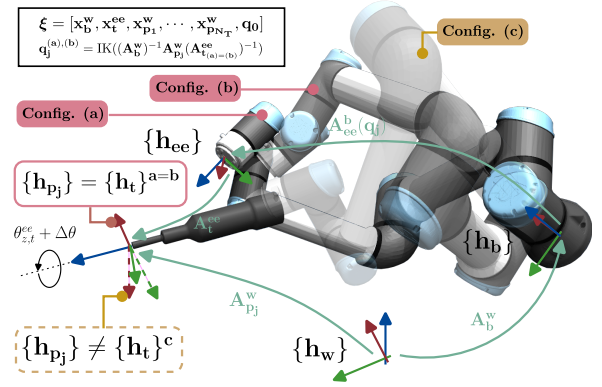


Fig. 2. Task redundancy problem. $\mathbf{q}_j^{(a)}$ and $\mathbf{q}_j^{(b)}$ are 2 out of 8 possible IK solutions for the case $\{\mathbf{h}_t\}^{a=b} = \{\mathbf{h}_{p_j}\}$. $\mathbf{q}_j^{(c)}$ is one of the solutions when applying an extra rotation $\Delta\theta \in [0; 2\pi]$ to the initial $\theta_{z,t}^{ee}$ of $\mathbf{x}_t^{ee}$.

$$\tau_j(\xi, \mathbf{q}_j) = \tau_g(\xi, \mathbf{q}_j) + \mathbf{J}^\top(\xi, \mathbf{q}_j) \mathbf{w}_j \quad (1)$$

where $\mathbf{q}_j$ is the robot configuration, ξ is the layout of the work-cell, τ_g is the contribution given by the gravitational field, while $\mathbf{J}$ is the robot Jacobian.

In a general case, ξ contains all the vectors $\mathbf{x}_k^i$ of the resources to be optimally placed in the environment, together with the robot initial configuration $\mathbf{q}_0$. To complete the task description, all the $\mathbf{q}_j$ must be computed. However, when applying $\mathbf{w}_j$, $\{\mathbf{h}_t\}$ must match $\{\mathbf{h}_{p_j}\}$: because ξ contains the poses of all the resources to be optimally organized, $\mathbf{q}_j$ is a function of the layout, and it can only be determined at a later stage. In the end, finding $\mathbf{q}_j(\xi)$ boils down to solving the IK of the manipulator, that allows to retrieve more solutions for a given pose (in the remainder, the dependence of $\mathbf{q}_j$ on ξ will be omitted for the sake of clarity).

This problem is even more challenging if the robot is redundant. Configurations (a) and (b) in Fig. 2 represent two different $\mathbf{q}_j$ corresponding to the case where $\{\mathbf{h}_t\}^{a=b}$ is superimposed to $\{\mathbf{h}_{p_j}\}$. Setting $\{\mathbf{h}_{p_j}\}$ helps in identifying the N_T targets, but every increment $\Delta\theta$ around the z axis of $\{\mathbf{h}_t\}$ would not change the outcome of the task, even if $\{\mathbf{h}_{p_j}\}$ and $\{\mathbf{h}_t\}$ ended up presenting a relative rotation around their shared z axis (task redundancy, configuration (c)). To account for this, we introduce $\bar{\mathbf{x}}_k^i = [\mathbf{t}_k^i, \theta_{x,k}^i, \theta_{y,k}^i]^\top \in \mathbb{R}^5$, to define the minimal set of coordinates for the operation.

The need to express $\mathbf{q}_j$ as a function of ξ is transversal to all robotic applications we target, and it can be solved as a *nested* optimization problem by leveraging the *leader-follower* paradigm [21], [22]. The problem becomes

$$\min_{\xi} \mathbf{f}_L(\xi, \mathbf{q}) \quad (2a)$$

$$\text{s.t.} \quad \mathbf{g}_L(\xi, \mathbf{q}) \leq \mathbf{0} \quad (2b)$$

$$\mathbf{q}_j \in \min_{\mathbf{q}_j} \mathbf{f}_{F_j}(\xi, \mathbf{q}_j) \quad (2c)$$

$$\text{s.t.} \quad \mathbf{g}_{F_j}(\xi, \mathbf{q}_j) \leq \mathbf{0} \quad (2d)$$

where $\mathbf{q} = [\mathbf{q}_1, \dots, \mathbf{q}_N]^\top \in \mathbb{R}^{\nu N}$ is the vector containing all the joint configurations, N is the number of followers, while $\xi \in \mathbb{R}^d$ are layout parameters to be optimized. We denote as $D = d + \nu N_T$ the total number of variables, with $\mathbf{f}_L : \mathbb{R}^D \rightarrow \mathbb{R}^{m_L}$ and $\mathbf{g}_L : \mathbb{R}^D \rightarrow \mathbb{R}^{k_L}$ the multi-objective

functions and the multi-objective inequality constraints for the *leader* problem, respectively. $\mathbf{f}_{\mathbf{F}_j} : \mathbb{R}^{d+\nu} \rightarrow \mathbb{R}^{m_{F_j}}$ and $\mathbf{g}_{\mathbf{F}_j} : \mathbb{R}^{d+\nu} \rightarrow \mathbb{R}^{k_{F_j}}$ are the objective functions and constraints for the followers, with $j \in \{1, \dots, N\}$.

Our framework can be employed for two types of optimization, according to the desired value of θ : if $\theta = 1$, the optimal solution is obtained for all the N_T targets at the same time; in case $\theta = 0$, an optimal solution is associated to each specific target. For this reason, the number of followers is computed as $N = \theta N_T + (1 - \theta)$, while $\bar{N} = N(1 - \theta)$ represents the number of times the optimization is repeated.

IV. METHODOLOGY

The most critical aspect of operations entailing static contacts is to guarantee the feasibility of the complete task. The hierarchical structure in (2) requires the leader to account for $\mathbf{q}$ during the resolution of ξ , but the former can only be obtained as a consequence of the latter.

Our proposal is to transform the complete problem into a iterative search towards the best solution, rather than solving the nested sub-problems with the traditional gradient-based techniques, given the difficulty in defining an analytical function that suitably catches the complex relations between the variables at stake. For what concerns the leader, we employed Bayesian Optimization, thanks to its capacity of generating layout candidates that are not only closer and closer to the best attainable result, but are also expected to improve the performances of the lower level optimizers. For the followers, the IK is solved through a normalizing flow, chosen for the possibility of overcoming the limitations of local or stochastic solvers, and propose stable solutions that help the leader in the learning phase.

In this work, we define the objective functions according to *primary* and *secondary* metrics, as a function of their impact on the application. While the former are directly linked to feasibility and performance, such as torque saturation margin and manipulability, the secondary metrics capture task-specific preferences, including path length minimization or joint centering inside the tolerated ranges. In the following, we will focus on the primary metrics for sub-problems (2a) and (2c), while we leave the presentation of secondary objectives to Section V, since they depend on the use-case.

A. Leader problem: torque saturation avoidance

Despite the possibility to keep the multi-objective formulation presented in (2), we preferred to define the leader problem with a scalar function f_L , as the weighted sum of a primary metric f_τ and the $m_L - 1$ *secondary* KPIs contained in $\mathbf{f}_{\mathbf{L}}^s$, scaled by w_τ and the diagonal matrix of weights $\mathbf{W}_{\mathbf{L}}^s$, respectively. The *leader* becomes

$$\min_{\xi} f_L(\xi, \mathbf{q}) = w_\tau f_\tau(\xi, \mathbf{q}) + \mathbf{W}_{\mathbf{L}}^s \mathbf{f}_{\mathbf{L}}^s(\xi, \mathbf{q}) \quad (3a)$$

$$\text{s.t. } \xi \in \Xi \quad (3b)$$

$$\mathcal{C}_0(\xi) = \{\emptyset\} \quad (3c)$$

where (3b) restricts the search space over a set of valid layouts Ξ , while (3c) is introduced to make sure that the group of resources presenting a collision remains empty.

To define the primary KPI f_τ , we introduce the normalized torques $\hat{\tau}_j = \Gamma^{-1} \mathbf{H}^{-1} \tau_j$, where τ_j is computed with (1), while $\mathbf{H}$ and Γ are the diagonal matrices containing the gear ratios and the maximum allowed torques, respectively. Given the need to account for N targets at the same time, we define the following primary objective for the leader (for readability, we drop the dependency on ξ and $\mathbf{q}_j$)

$$f_\tau = \frac{1}{N} \sum_{j=1}^N \|\hat{\tau}_j\|_2 = \frac{1}{N} \sum_{j=1}^N \sqrt{\sum_{k=1}^{\nu} \hat{\tau}_{j,k}^2} \quad (4)$$

Once the optimization is over, the optimal value f_τ^* encodes a mean margin to saturation, representative of all the targets and accounting for all the ν joints at the same time. We considered this metric to be the most suitable for our case because it adapts more easily to the types of optimization that can be handled by our framework. More specifically, when a unique optimal solution for the work-cell is desired ($\theta = 1$), selecting $f_\tau = \max_j (\max_k (\hat{\tau}_{j,k}))$ would lead to results that are too conservative and oriented to one of the specific targets, in contrast with the aim of the optimization. Instead, for sequential solutions ($\theta = 0$), the number of followers N is unitary for each one of the $\bar{N} = N_T$ repetitions of the framework, and defining f_τ as in (4) helps in finding a margin that is representative of all the joints.

B. Follower problems: inverse manipulability minimization

With reference to (2), each of the N problems formulated as constraint (2c) is responsible for determining the best joint configuration $\mathbf{q}_j$. For the generic follower represented by (2c), we define a scalar function f_{F_j} , $j \in \{1, \dots, N\}$, as the weighted sum of a primary metric f_{δ_j} and the $m_{F_j} - 1$ *secondary* KPIs contained in $\mathbf{f}_{\mathbf{F}_j}^s$ (w_{δ_j} and $\mathbf{W}_{\mathbf{F}_j}^s$ have the same meaning as w_τ and $\mathbf{W}_{\mathbf{L}}^s$ in (3a)). Therefore, each one of the N followers can be defined as

$$\min_{\mathbf{q}_j} f_{F_j}(\xi, \mathbf{q}_j) = w_{\delta_j} f_{\delta_j}(\xi, \mathbf{q}_j) + \mathbf{W}_{\mathbf{F}_j}^s \mathbf{f}_{\mathbf{F}_j}^s(\xi, \mathbf{q}_j) \quad (5a)$$

$$\text{s.t. } \mathbf{q}_{j,L} \leq \mathbf{q}_j \leq \mathbf{q}_{j,U} \quad (5b)$$

$$\bar{\mathbf{x}}_t^w(\mathbf{q}_j) = \bar{\mathbf{x}}_{p_j}^w \quad (5c)$$

$$\mathcal{C}(\xi, \mathbf{q}_j) = \emptyset \quad (5d)$$

where (5b) imposes upper (U) and lower (L) bounds on the joints. Instead, (5c) relates $\bar{\mathbf{x}}_t^w$, obtained from $\mathbf{A}_b^w \mathbf{A}_{ee}^b(\mathbf{q}_j) \mathbf{A}_t^{ee}$ as described in Section III, to the 5 minimal coordinates of the target location (θ_{z,p_j}^w is redundant). (5d) is enforced to avoid self-collisions of the manipulator and collisions between the robot, configured as $\mathbf{q}_j$, and the environment, organized according to ξ . In our case, we solved the IK considering the inverse manipulability as the primary KPI

$$f_{\delta_j} = \frac{1}{\sqrt{\det(\mathbf{J}(\xi, \mathbf{q}_j) \mathbf{J}^T(\xi, \mathbf{q}_j))}} \quad (6)$$

C. Optimization of the leader problem

The main complexity of the problem under exam lies in the difficulty in expressing the relations between variables through analytical functions and gradients. In this regard, the choice of defining f_L as in (3a) gives a meaningful metric to

Algorithm 1: Proposed optimization framework

```

Given:  $N_{it}, t, d, N, \bar{N}, \mathbf{W}_L^s, \mathbf{W}_F^s = [\mathbf{W}_{F_1}^s, \dots, \mathbf{W}_{F_N}^s], \Xi$ 
1  $\xi^*, \mathbf{q}^*, \mathbf{f}_L^* \leftarrow [];$ 
2 for  $r \leftarrow 1$  to  $\bar{N}$  do //  $\bar{N} = 1$  if  $\theta = 1$ ;  $\bar{N} = N_T$  if  $\theta = 0$ 
3    $\bar{\xi}, \bar{\mathbf{q}}, \bar{\mathbf{f}}_L, \bar{\mathbf{f}}_\tau \leftarrow [];$ 
4   for  $i \leftarrow 1$  to  $N_{it}$  do
5      $\bar{\mathbf{q}}^*, \mathbf{f}_L, \mathbf{f}_\tau \leftarrow [];$   $\xi_c \leftarrow \text{GetCandidates}(\Xi);$  // eq.(3b)
6     for  $k \leftarrow 1$  to  $t$  do
7        $\xi \leftarrow \xi_c[k, :];$   $\mathbf{F}_L \leftarrow [];$ 
8        $C_0 \leftarrow \text{GetCollisions}(\xi);$  // Simulator
9       if  $C_0 = \{\emptyset\}$  then // No collisions; eq.(3c)
10         $\bar{\mathbf{q}}^* \leftarrow [];$ 
11        for  $j \leftarrow 1$  to  $N$  do
12           $\mathbf{q}_j^* \leftarrow \text{OptimizeConfigs}(\xi, \mathbf{W}_F^s[j]);$ 
13          if  $\mathbf{q}_j^* \neq \mathbf{0}$  then
14             $\mathbf{F}_L[j, :] \leftarrow \text{GetParameters}(\xi, \mathbf{q}_j^*);$ 
15          else // Unfeasible IK penalty
16             $\mathbf{F}_L[j, :] \leftarrow \infty;$ 
17           $\bar{\mathbf{q}}^*[j] \leftarrow \mathbf{q}_j^*;$ 
18           $f_\tau \leftarrow \sum_{i=1}^N \sqrt{\|\mathbf{F}_L[i, 1:\nu]\|_2} / N;$  // eq.(4)
19           $\mathbf{f}_L^s \leftarrow \text{GetSecondaryKPIs}(\mathbf{F}_L[:, (\nu+1):]);$ 
20           $\bar{\mathbf{q}}^*[k] \leftarrow \bar{\mathbf{q}}^*; \mathbf{f}_L[k] \leftarrow w_\tau f_\tau + \mathbf{W}_L^s \mathbf{f}_L^s;$  // (3a)
21           $\bar{\mathbf{f}}_\tau[k] \leftarrow f_\tau;$ 
22        else // Collision penalty
23           $\mathbf{f}_L[k] \leftarrow \infty; \bar{\mathbf{f}}_\tau[k] \leftarrow \infty;$ 
24       $\bar{\xi}[i], \bar{\mathbf{q}}[i], \bar{\mathbf{f}}_L[i] \leftarrow \text{UpdateBestSoFar}(\xi_c, \bar{\mathbf{q}}^*, \mathbf{f}_L);$ 
25       $\bar{\mathbf{f}}_\tau[i] \leftarrow \min(\bar{\mathbf{f}}_\tau);$  // Quantity for the box plot
26       $\xi^*[r], \mathbf{q}^*[r], \mathbf{f}_L^*[r] \leftarrow \bar{\xi}, \bar{\mathbf{q}}, \bar{\mathbf{f}}_L;$  //  $\theta = 1: \xi^* = \bar{\xi}, \mathbf{f}_L^* = \bar{\mathbf{f}}_L$ 
27 return  $\xi^*, \mathbf{q}^*$ 

```

evaluate the outcome of the process, but does not represent the real analytical function, which is not known.

To solve (3), we resorted to Bayesian Optimization (BO) [23]. This model-based technique introduces a Gaussian Process $\mathcal{GP}(\mu(\xi), k(\xi, \xi'))$ that leverages iterative observations of the process through $f_L(\xi, \mathbf{q})$, in order to assume a prior distribution over the unknown underlying function ($\mu(\cdot)$ and $k(\cdot, \cdot)$ are the mean and the variance, respectively). BO allows to transform (3a) into the following iterative procedure (i is the iteration counter)

$$\xi_{i+1} = \underset{\xi \in \mathcal{X}}{\operatorname{argmin}} \eta(\xi) \quad (7)$$

with $\mathcal{X}$ identifying a domain where (3b) and (3c) are both verified. $\eta(\xi)$ is the *acquisition function*, that quantifies the utility of evaluating f_L at $(\xi, \mathbf{q})$, and depends only on $\mu(\xi)$ and $k(\xi, \xi')$. Based on ξ_{i+1} and the corresponding $\mathbf{q}_{i+1}$ resulting from the followers, an observation $y_{i+1} = f_L(\xi_{i+1}, \mathbf{q}_{i+1})$ is obtained, and the $\mathcal{GP}$ is fitted on the augmented dataset $\mathcal{D}_{i+1}^{\text{BO}} = \{\mathcal{D}_i^{\text{BO}}, (\xi_{i+1}, y_{i+1})\}$. With this strategy, the effect of $\mathbf{q}_{i+1}$ is embedded in y_{i+1} and, after updating mean and covariance, (7) allows to suggest new candidates that will converge to the optimal value ξ^* .

BO is quite effective when the size d of the search space is limited, but it struggles when the number of variables is considerable, like in our case. For this reason, we employed TuRBO [24], where a collection of m $\mathcal{GP}$ s is maintained at the same time, and restricted to a specific trust region $\mathcal{T}_k$, $k \in \{1, \dots, m\}$. Each candidate ξ_i^j of the batch composed of t individuals (with $j \in \{1, \dots, t\}$) can be obtained by modifying (7) as follows

Algorithm 2: OptimizeConfigs

```

Given:  $\xi, \mathbf{q}_L, \mathbf{q}_U, \mathbf{W}^s, w_\delta, N_d, N_s$ 
1  $\mathbf{A}_b^w, \mathbf{A}_{p_j}^w, \mathbf{A}_{ee}^w \leftarrow \text{Unpack}(\xi);$   $\mathbf{t}_t^{ee}, \phi_t^{ee} \leftarrow \text{Unpack}(\mathbf{A}_{ee}^w);$ 
2  $\theta_{x,t}^{ee}, \theta_{y,t}^{ee}, \theta_{z,t}^{ee} \leftarrow \phi_t^{ee};$   $f \leftarrow \infty; f^* \leftarrow \infty; \mathbf{q}^* \leftarrow \mathbf{0}; \mathbf{q}_j \leftarrow [];$ 
3 for  $i \leftarrow 1$  to  $N_d - 1$  do
4    $\Delta\theta \leftarrow \frac{2\pi}{N_d};$   $\phi_t^{ee} \leftarrow [\theta_{x,t}^{ee}, \theta_{y,t}^{ee}, \theta_{z,t}^{ee} + i \cdot \Delta\theta]^\top;$ 
5    $\mathbf{R}_t^{ee} \leftarrow \text{Eul2Rotm}(\phi_t^{ee});$   $\mathbf{A}_t^{ee} \leftarrow [\mathbf{R}_t^{ee}, \mathbf{t}_t^{ee}; \mathbf{0}, 1];$ 
6    $\mathbf{A}_{ee}^b \leftarrow (\mathbf{A}_b^w)^{-1} \mathbf{A}_{p_j}^w (\mathbf{A}_t^{ee})^{-1};$  // eq.(5c) inverted
7    $\mathbf{q}_j[i] \leftarrow \text{ikflow}(\mathbf{A}_{ee}^b, N_s);$  // eq.(5b) is enforced here
8 if  $\mathbf{q}_j \neq []$  then // Feasible IK; otherwise  $\mathbf{q}^*$  remains =  $\mathbf{0}$ 
9   for  $i \leftarrow 1$  to  $|\mathbf{q}_j|$  do
10     $N_c \leftarrow \text{GetCollisions}(\xi, \mathbf{q}_j[i]);$  // Simulator
11     $f_\delta \leftarrow 1 / \sqrt{\det(\mathbf{J}_j^\top \mathbf{J}_j)};$ 
12     $\mathbf{f}^s \leftarrow \text{GetSecondaryKPIs}(\mathbf{q}_L, \mathbf{q}_U, \mathbf{q}_j[i]);$ 
13     $f \leftarrow w_\delta f_\delta + \mathbf{W}^s \mathbf{f}^s;$  // eq.(5a)
14    if  $(f < f^*) \wedge (N_c = 0)$  then // eq.(5d)
15       $f^* \leftarrow f; \mathbf{q}^* \leftarrow \mathbf{q}_j[i];$ 
16 return  $\mathbf{q}^*$ 

```

$$\xi_{i+1}^j = \underset{k \in \{1, \dots, m\}}{\operatorname{argmin}} \underset{\xi \in \mathcal{T}_k}{\operatorname{argmin}} \bar{f}_k^j(\xi) \quad (8)$$

The term $\bar{f}_k^j(\cdot) \sim \mathcal{GP}_k^i(\mu_k(\cdot), k_k(\cdot, \cdot))$ is a posterior sample that models plausible realizations of f_L , restricted to $\mathcal{T}_k^i$. Equation (8) is solved for each component of the batch, until $\{\xi_{i+1}^j\}_{j=1}^t$ is obtained.

The mechanism that relies on the propagation of $\mathcal{T}_k$ is the core reason behind our choice to employ TuRBO over other methods. More specifically, the observations $y_i = f_L(\xi_i, \mathbf{q}_i)$ allow to update the dataset for the learning process by directly accounting for the layouts, while also indirectly encoding pieces of information concerning $\mathbf{q}_i$. As a consequence, the information about promising regions for the optimality of the followers is embedded in the candidates ξ already during the resolution of the leader problem. Based on this, the convergence speed of the overall bi-level structure is increased, provided that a stable and global resolution technique is selected for the followers.

Alg. 1 shows the overall framework. The methods written in teletype are meant to abstract the presented equations, making the pseudo-code capable of representing several resolution schemes used as benchmarks in Subsection V-A, and not just TuRBO. The function `OptimizeConfigs` is explained more in detail in Subsection IV-D through Alg. 2.

D. Optimization of the follower problems

Problem (5) entails the robot IK resolution. For a certain pose $\mathbf{p} \in SE(3)$, the IK can be defined as the map $f: \mathbf{p} \mapsto \mathbf{q} \in \mathbb{R}^\nu$. In general, the set of joint configurations forms a high-dimensional manifold, characterized by joint limits, self-collisions and kinematic redundancy.

As anticipated in Subsection IV-C, avoiding local optimal results in the followers has indirect benefits during the generation of the layout candidates ξ . In order to remove from the process the dependence on initial conditions, we devised a gradient-free approach that does not involve the traditional optimization techniques to solve (5). More specifically, we fine-tuned `ikflow` [19], which is based on a conditional *normalizing flow* g_θ , representing the mapping $\mathbf{q} = g_\theta(\mathbf{z}, \mathbf{p})$.

The quantity $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_\nu)$ is a *random latent variable* that is sampled from a simple base distribution. Thanks to the *invertibility* property, after learning the map $\mathbf{z} = g_\theta^{-1}(\mathbf{q}, \mathbf{p})$, it is possible to sample variables $\mathbf{z}$ from the known space $\mathcal{N}$, and obtain a vector $\mathbf{q} = g_\theta(\mathbf{z}, \mathbf{p})$ that is distributed like the unknown joint solutions for the pose $\mathbf{p}$. In our case, we tackled the resolution of (5) by devising the procedure implemented in Alg. 2, after training the normalizing flow on custom datasets $\mathcal{D}^{\text{IK}} = \{(\mathbf{q}_i, \mathbf{p}_i)\}_{i=1}^{\bar{N}}$, specific for each robot ($\bar{N} = 2 \cdot 10^6$). The learned map g_θ^{-1} relates $\mathbf{q}$ to $\mathbf{x}_{\text{ee}}^{\text{b}}$: for each target location j , we define a new pose $\mathbf{x}_{\text{ee}}^{\text{b}}$ associated to $\mathbf{A}_{\text{ee}}^{\text{b}}$ (step 6), we sample N_s latent variables $\mathbf{z}$ and we feed them to `ikflow` to retrieve N_s variables $\mathbf{q}_j$ (step 7). Since sampling $\mathbf{q}_j$ takes only some milliseconds, kinematic redundancy can be handled efficiently as a function of N_s : for $N_s \rightarrow \infty$, all the possible $\mathbf{q}_j$ are retrieved by conditioning $\mathbf{z}$ on $\mathbf{x}_{\text{ee}}^{\text{b}}$ and feeding the normalizing flow in the opposite direction. Task redundancy is treated separately, after discretizing the possible rotations $\Delta\theta = 2\pi/N_d$ around the z axis shared by $\{\mathbf{h}_t\}$ and $\{\mathbf{h}_{p_j}\}$: in the limit, for $N_d \rightarrow \infty$, all the possible rotations $\theta_{z,t}^{\text{ee}} + \Delta\theta$ would be accounted for.

V. CASE STUDY

To validate our method, we tested the framework with two different use cases. In the first one, we considered $d = 14$ optimization variables in a screw-tightening scenario, while we settled for a smaller size of the design vector in the second use case ($d = 3$), that concerns a drilling task.

Given the design stage for which our framework is engineered, we conducted all the experiments in simulation, relying on a laptop featured with 32 GB of RAM and a Intel(R) Core(TM) i9-14900HX processor operating at 2.20 GHz. Given the need to include the machine learning structures presented in Subsections IV-C and IV-D we implemented all the codes in Python. Additionally, we wrapped all the algorithms around the Python bindings of MuJoCo [25], which allowed to enforce the collision checks and retrieve the main process quantities at each iteration.

A. Benchmark strategies

Motivated by the absence of available similar frameworks in the literature, we tested the validity of our proposal by implementing the four different strategies in Table I. Since the first use case is the most demanding in terms of computational resources, the results of the benchmark are presented only for that application. For the second use case, our framework will be compared to an experience-based solution instead. While (a) is based on the schemes presented previously, (b) relies on the state-of-the-art genetic algorithm `cma-es` [26]. We chose this method due to the similarity with `TuRBO` for what concerns the *exploration-exploitation* tradeoff. The mean μ_0 and the exploration σ_0 are taken as visible in Table I, in order to be consistent with the units of measure of the variables involved.

The beneficial effects of `ikflow` for the optimization framework we setup are demonstrated through strategy (c), where Alg. 2 is substituted by a custom solution that takes

inspiration from the memetic solver `bioik2` [20]. In this case, we solved (5a) by minimizing the total cost

$$J = \|w_m C_m + w_l C_l + w_p C_p + w_c C_c\|_2 \quad (9)$$

The first two terms are the same primary and secondary KPIs presented with (6) and (11), while the two extra costs are added to introduce (5c) and (5d) as soft constraints. More in detail, $C_p = |\bar{\mathbf{x}}_t^{\text{w}}(\mathbf{q}_j) - \bar{\mathbf{x}}_{p_j}^{\text{w}}|$ and $C_c = \sum_c (1000d_c + 1)$, with c the number of elements in collisions, while d_c is the penetration distance. The weights $w_p = 10^8$ and $w_c = 10^5$ are orders of magnitude higher than the others ($w_m = w_l = 10^2$), in order to prioritize C_p and C_c in (9). The population size N_{pop} and the number of elite individuals N_e were selected as in Table I, so that the error on the tool pose was comparable with the results of `ikflow`.

Strategy (d) enforces the `random benchmark`. With this, we aimed at simulating the choices of $N_{it} \cdot t$ people, that are asked to create their own layout. In this case, we neglected the resolution of the task redundancy ($N_d = 1$), imposing each frame $\{\mathbf{h}_t\}$ to match exactly $\{\mathbf{h}_{p_j}\}$.

TABLE I
BENCHMARK STRATEGIES IMPLEMENTED.

Str.	Name	Leader	Followers	Parameters
(a)	TuRBO _{ikf}	TuRBO	ikflow	$m = 5, N_d = 90, N_s = 25$
(b)	cma-es _{ikf}	cma-es	ikflow	$\mu_0 = \mathbf{0}, \sigma_0 = \mathbf{2}, N_d, N_s \rightarrow \text{(a)}$
(c)	TuRBO _{bio}	TuRBO	bioik2	$N_{\text{pop}} = 150, N_e = 25, m \rightarrow \text{(a)}$
(d)	random	random	ikflow	$N_d = 1, N_s = 1$

B. Use case I - screw-tightening application

The first use case concerns the optimization of the layout for the assembly of the *bench vise* in Fig. 4. We aimed at finding a tradeoff layout ($\theta = 1, N = 4, \bar{N} = 1$), and the target locations denote the screws. We considered constant values for the wrenches, and we supposed that the only non-zero components in $\mathbf{w}_j$ were $F_{z,j}$ and $M_{z,j}$, presenting equal values for all the screws ($|F_z| = 30 \text{ N}, |M_z| = 20 \text{ Nm}$).

The selected robot is a Universal Robots UR5e ($\nu = 6$), while the tool weighs 1.5 kg. Since $\mathbf{x}_t^{\text{ee}}$ is part of ξ , the shape of the support connecting the tool to the flange can only be obtained once the optimization is over (the same reasoning holds for the support connecting the robot base to the floor), and we neglected the effect of its mass on τ_g . To reduce the computational burden for the algorithm, we defined a single frame $\{\mathbf{h}_p\}$ in the center of the bench vise: because all matrices $\mathbf{A}_{p_j}^{\text{p}}$ are constant, the required $\mathbf{A}_{p_j}^{\text{w}}$ were easily computed at each iteration. The optimization vector we considered is $\xi = [x_b^w, y_b^w, \theta_{x,b}^w, x_t^{\text{ee}}, y_t^{\text{ee}}, \theta_{x,t}^{\text{ee}}, x_p^w, y_p^w, \mathbf{q}_0^{\text{T}}]^{\text{T}} \in \mathbb{R}^{14}$.

1) *Leader secondary KPI - Minimum path metric*: The presence of $\mathbf{q}_0$ in ξ can be explained by the secondary KPI, that contributes in the minimization of the cycle time by reducing the length of the complete closed path to execute the task. Therefore, we define a single secondary KPI in (3a)

$$f_L^s = \sum_{i=1}^{N_P} \sum_{k=1}^{N_{P_k}} |\mathbf{p}_{k+1}^i - \mathbf{p}_k^i| / \bar{L} \quad (10)$$

where $N_P = N_T + 1$ corresponds to the number of paths in a sequence (*i.e.*, all paths between the N_T targets plus the

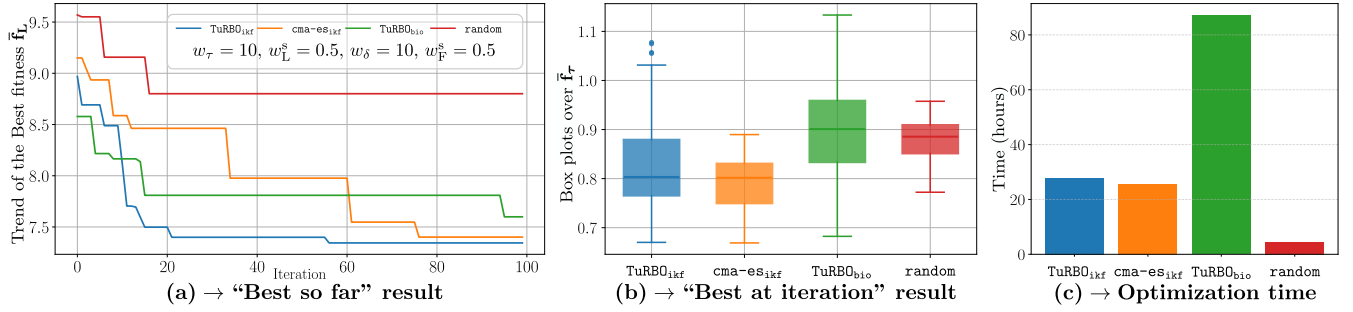


Fig. 3. Results for Use case I. The meaning of $\bar{f}_L$ and $\mathbf{f}_L$ is visible in Alg. 1. (a): Trends of the best fitness for $N_{it} = 100$ iterations and $t = 40$ individuals. (b): box plots of the best results at each iteration (not the global best). (c): Time taken by the 4 strategies in the optimization.

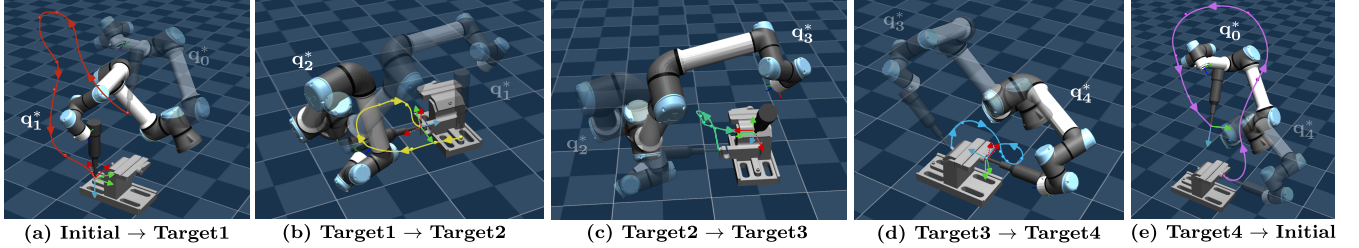


Fig. 4. Optimal results for the screw-tightening application. In each sub-figure, the path given by RRTConnect is reported. The blurred configurations are the starting ones, while the visible configurations are the values of $\mathbf{q}_j^*, j \in \{1, \dots, 4\}$ (Fig. (a) $\rightarrow$ (d)). (e): optimal initial configuration $\mathbf{q}_0$. For the reference frames, the solid lines denote the targets $\{\mathbf{h}_{p_j}\}$, while dashed ones represent the optimal results for $\{\mathbf{h}_t\}$ given by ikflow.

initial and final paths from and to $\{\mathbf{h}_{ee}^0\}$). Instead, N_{P_k} is the number of points in each path, and we introduce $|\cdot|$ to denote the length of the segment between consecutive points $\mathbf{p}_k^i$, whose number is not fixed and is given by a custom implementation of the RRTConnect algorithm [27]. Finally, $\bar{L} = 2\pi\bar{R}$ is a heuristic value to account for the path length with respect to the robot size, with $\bar{R}$ denoting the maximum robot reach ($\bar{R} = 0.85$ m).

2) Followers secondary KPI - Joint ranges centering:

The joints configuration obtained at step 16 in Alg. 2 may be characterized by elements that are too close to their limits. For this purpose, we defined the secondary objective as

$$f_F^s = (\mathbf{q} - \mathbf{m})^\top \mathbf{T} (\mathbf{q} - \mathbf{m}) \quad (11)$$

where $\mathbf{m} = (\mathbf{q}_L + \mathbf{q}_U)/2 \in \mathbb{R}^\nu$ is the vector of midpoints, while $\mathbf{T} = \mathbf{C}^{-\top} \mathbf{C}^{-1} \in \mathbb{R}^{\nu \times \nu}$, with $\mathbf{C} = \text{diag}((\mathbf{q}_L - \mathbf{q}_U)/2) \in \mathbb{R}^\nu$ is the diagonal matrix of the half-ranges.

3) *Results*: The performances of the presented methods are visible in Fig. 3. In particular, Fig. 3a shows the *global* behavior of the strategies, by representing the trends of the best fitness function $\bar{f}_L$, obtained with UpdateBestSoFar at step 24 of Alg. 1. Although (a) and (b) achieve similar results in the minimization of the fitness function, TuRBO_{ikf} takes only 57 iterations to reach the best solution, while cma-es_{ikf} takes 76. It must be noted that BO alone would not be enough to explore the search space adequately: looking at the results for strategy (c), where the followers are solved by means of bioik2, the procedure presents a flat region that suggests a stagnation stage during the learning process. In the end, strategy (d) shows the worst global performances, showing that proposing a total amount of $N_{it} \cdot t = 4000$ different layouts is not enough to find solutions that can be compared with the results of optimization algorithms.

The global information alone is not enough to claim the validity of the proposed framework. For this reason, we corroborate the analysis with the *local* information contained in Fig. 3b. The box plots are based on $\bar{f}_\tau$ (step 25 of Alg. 1), that is a dataset of $N_{it} = 100$ values, where each element is the smallest f_τ obtained by one among the $t = 40$ individuals at a specific iteration (we consider only the primary KPI since it is the most influent, according to the selected weights). Consequently, each element of the dataset is not related to the best fitness registered until that moment, and represents the capacity of the algorithms in exploring the search space. Despite (a) and (b) settle for the same median value, the box plot for the former indicates that TuRBO tends to explore the search space in a more insightful manner, thanks to the mechanism of fitting Gaussian models ($\mathcal{GP}$ s) in trust regions, that are started and stopped as a function of subsequent improvements or failures. This fact is also confirmed by the presence of some outliers. In fact, every new candidate is a tradeoff between exploitation of the current results and exploration of new regions: in case one of them turns out to be a bad suggestion, its corresponding trust region gets promptly stopped. The observation of the process through f_L allows TuRBO to avoid generating layouts in areas where an outlier was previously generated, in favor of regions that lead to better results. As expected, the model-based exploration of TuRBO allows this method to be more sample-efficient and, therefore, more suitable to be employed as the solver for the leader problem: the smaller box plot for cma-es_{ikf} denotes a smaller capability of this model-free method in exploring promising regions of the search space, suggesting more often candidates that are quite similar to each other. On the contrary, the box plot for (c) helps in

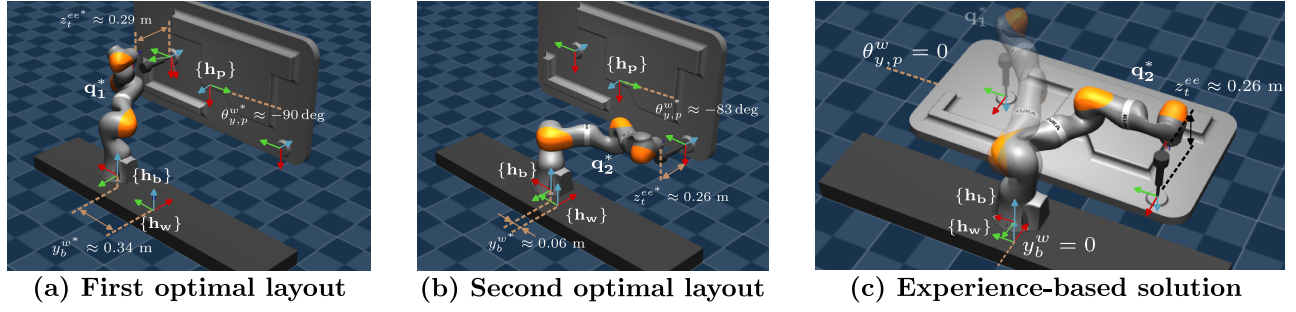


Fig. 5. Optimal results for the second use case. (a) → (b): ξ^* and q^* obtained sequentially with Alg. 1, imposing $\bar{N} = N_T = 2$. (c): Reasonable experience-based proposal. The robot base, the tool and the piece are placed in fixed positions: without positioning modules, a tradeoff solution is found.

explaining the stagnation phase: for similar layouts suggested by TuRBO, the memetic algorithm is not able to return a stable result, preventing the $\mathcal{GP}_s$ to be fitted on consistent datasets. The span of the Whisker plot for (c) is the largest, since TuRBO is not able to infer directions that could be beneficial for the followers.

The total time required by the strategies is visible in Fig. 3c. The best tradeoff between attained performances and computational effort is surely represented by (a), since it needs only ≈ 2 hours more than (b) to complete the process (in reality, (a) requires only 21 iterations to reach the best result achieved by `cma-esikf` at iteration 75, so we can say that TuRBO_{ikf} is globally better).

Table II shows the most important parameters for strategies (a) and (b) ((c) and (d) cannot be compared with them, since they result in sub-optimal solutions). Together with the inverse manipulability f_δ^* , we reported the metric $\lambda^* = \min(1/\hat{\tau}_{j,1}^*, \dots, 1/\hat{\tau}_{j,\nu}^*)$, which quantifies the smallest amplification factor that can be applied to the wrench before reaching saturation. The resulting values are all ≥ 1 , meaning that it would be possible to increase the applied wrench before reaching saturation of at least one of the actuators. Instead, to quantify the results of the secondary KPI for the followers in (11), we introduced $\Delta q\% = [\nu - \mathbf{C}^{-1}(|q^* - \mathbf{m}|)]/\nu$ (unitary in case all joints are perfectly centered in the ranges). As it can be seen from the small weights for the secondary KPIs in Fig. 3a, (a) is not always able to guarantee the best centering in the joint limits, despite producing results that are always higher than 0.5, unlike (b). In terms of f_δ^* , both strategies reach similar and acceptable values, confirming the stability of the procedure proposed in Alg. 2. In general, the strategy we chose for our framework achieves better results than `cma-esikf`, as demonstrated by the number of colored cells.

In the end, Fig. 4 shows the best results obtained by TuRBO_{ikf}. Images (a) → (e) show the optimal path computed through RRTConnect: despite most of the targets present orthogonal orientations, the planner is always capable of finding collision-free paths connecting them. Moreover, the algorithm is correctly suggesting to place the robot on the same side as the second screw, that is the only one not positioned on a symmetry plane, and can be successfully reached by the robot from a smaller range of feasible positions.

TABLE II
SCALING FACTORS AND FOLLOWER METRICS FOR USE CASE I.

Screw	Our framework (St. (a))			St. (b)		
	λ^*	f_δ^*	$\Delta q\%$	λ^*	f_δ^*	$\Delta q\%$
1	2.04	9.80	0.64	1.88	11.81	0.70
2	4.35	86.35	0.53	4.66	92.88	0.42
3	1.85	11.37	0.50	1.78	13.68	0.74
4	1.98	13.56	0.64	1.78	15.22	0.44

C. Use case II - drilling application

In the second use case we focused on the realization of $N_T = 2$ holes on the aluminum plate visible in Fig. 5. In a drilling task, the thrust force is usually higher than in screwing, and we selected $|F_z| = 150$ N and $|M_z| = 10$ Nm for both the target locations. We employed a Kuka LBR iiwa 14 R820 robot, that can withstand higher wrenches than a UR5e and is featured with $\nu = 7$ DOFs. The considerable distance between the holes to be drilled (≈ 1 m) resulted in the necessity to introduce a linear positioning system for the robot base, with a span of ± 1 m with respect to $\{h_w\}$. We assumed availability of two additional auxiliary modules: one to regulate the orientation of the piece $\theta_{y,p}^w \in \{-90; 0\}$ deg, while the second to regulate $z_t^{ee} \in \{0.26; 0.41\}$ m. In this second use case, we performed the optimization to obtain a sequence of layouts and joint configurations ($\theta = 0 \rightarrow N = 1, \bar{N} = 2$), instead of a single tradeoff solution. Motivated by the results presented in Subsection V-B.3, we performed the sequential optimization employing only TuRBO_{ikf}, and the results are compared with an experience-based solution, where no positioning module is available (see Fig. 5c). The considered optimization vector is $\xi = [y_b^w, z_t^{ee}, \theta_{y,p}^w]^T \in \mathbb{R}^3$.

1) *Secondary KPIs - minimum distance:* For the leader we considered $f_L^s = |y_b^w - y_{b,prev}^w|$. The variable $y_{b,prev}^w$ accounts for the last optimal result of y_b^w contained in $\xi^*[r]$ at step 26 in Alg. 1. For the first optimal layout ($r = 1$) we considered $y_b^w = 0$, while for the second we imposed $y_b^w = 0.34$ m, that is the optimal solution found for $r = 1$. For the followers, we employed the same KPI as in the previous case.

2) *Results:* The overall process took ≈ 5.36 h, and we solved the kinematic redundancy by increasing $N_s = 2000$ samples, while we kept the same resolution $N_d = 90$. Fig. 5a and 5b show the layouts and robot configurations we obtained. In both cases, the algorithm settles for a condition where the aluminum plate is rotated vertically, since the

robot presents lower values of inverse manipulability for this orientation, if compared to a situation where no positioning module is available and task redundancy is not accounted for (Fig. 5c: for both holes, we found $\mathbf{q}_1^*$ and $\mathbf{q}_2^*$ solving only the kinematic redundancy and imposing $\{\mathbf{h}_t\}$ coincident to $\{\mathbf{h}_{p_j}\}$). This fact is also visible in Tab. III: in terms of scaling factors λ^* and manipulability f_δ^* , our framework attains better performances than a solution that is imposed by the system integrators, without relying on a real optimization procedure. Instead, also for the second use case, the weights we selected for the secondary KPI expressed with (11) and quantified by $\Delta q\%$, are not enough to guarantee the best centering in the limits; however, the random values of this metric in the second scenario are the result of choices that do not target them specifically, while in the optimized scenario the algorithm is choosing to sacrifice some optimality for the joint centering, since it is the least relevant term in (5a).

TABLE III
SCALING FACTORS AND FOLLOWER METRICS FOR USE CASE II.

Hole	Our framework (St. (a))			Experience-based		
	λ^*	f_δ^*	$\Delta q\%$	λ^*	f_δ^*	$\Delta q\%$
1	3.04	12.69	0.49	1.57	14.00	0.82
2	3.33	6.46	0.62	1.96	7.15	0.78

VI. CONCLUSIONS AND FUTURE WORKS

This work presents an optimization framework for the design of robotic work-cells for static contact-based operations. By formalizing the problem according to the leader–follower paradigm and introducing an efficient exploration of redundant configurations, the method allows for the joint optimization of several design variables. The results confirm that addressing task redundancy and layout optimization in a unified bi-level framework can significantly enhance both the performance and robustness of robotic applications, while removing the reliance on trial-and-error practices. Future work will focus on extending the methodology to general contact-based robotic tasks, such as milling or polishing, where the range of motion is no longer negligible, and they cannot be treated as static.

REFERENCES

- [1] M. Suomalainen, Y. Karayiannidis, and V. Kyrki, “A survey of robot manipulation in contact,” *Robotics and Autonomous Systems*, vol. 156, p. 104224, 2022.
- [2] T. Olsson, M. Haage, H. Kihlman, R. Johansson, K. Nilsson, A. Robertsson, M. Björkman, R. Isaksson, G. Ossbahr, and T. Brogårdh, “Cost-efficient drilling using industrial robots with high-bandwidth force feedback,” *Robotics and Computer-Integrated Manufacturing*, vol. 26, no. 1, pp. 24–38, 2010.
- [3] A. A. Malik and A. Brem, “Digital twins for collaborative robots: A case study in human-robot interaction,” *Robotics and Computer-Integrated Manufacturing*, vol. 68, p. 102092, 2021.
- [4] Z. Shiller, “Optimal robot motion planning and work-cell layout design,” *Robotica*, vol. 15, no. 1, pp. 31–40, 1997.
- [5] M. Tay and B. Ngoi, “Optimising robot workcell layout,” *The International Journal of Advanced Manufacturing Technology*, vol. 12, no. 5, pp. 377–385, 1996.
- [6] J. Baumgärtner, A. Puchta, and J. Fleischer, “Beyond proxies: A direct time-optimal approach to robot cell layout optimization,” *CIRP Annals*, 2025.

- [7] R. Ruzarovsky, T. Horak, R. Bocak, M. Csekei, and R. Zelník, “Integrating energy and time efficiency in robotic manufacturing cell design: A methodology for optimizing workplace layout,” *Machines*, vol. 13, no. 1, p. 38, 2025.
- [8] P. Franceschi, S. Mutti, and N. Pedrocchi, “Optimal design of robotic work-cell through hierarchical manipulability maximization,” *Robotics and Computer-Integrated Manufacturing*, vol. 78, p. 102401, 2022.
- [9] M. Ferre, P. E. Peralta, V. R. Gallego, N. Barbosa, and M. Á. Sánchez-Urán, “Forces analysis on robotics screwing tasks,” in *Iberian Robotics conference*. Springer, 2023, pp. 553–563.
- [10] H. Tugal, K. Cetin, Y. Petillot, M. Dunnigan, and M. S. Erden, “Manipulation at optimum locations for maximum force transmission with mobile robots under environmental disturbances,” *Autonomous Robots*, vol. 46, no. 6, pp. 769–782, 2022.
- [11] G. Li, W. Zhu, H. Dong, and Y. Ke, “A method for robot placement optimization based on two-dimensional manifold in joint space,” *Robotics and Computer-Integrated Manufacturing*, vol. 67, p. 102002, 2021.
- [12] S. Mitsi, K.-D. Bouzakis, D. Sagris, and G. Mansour, “Determination of optimum robot base location considering discrete end-effector positions by means of hybrid genetic algorithm,” *Robotics and Computer-Integrated Manufacturing*, vol. 24, no. 1, pp. 50–59, 2008.
- [13] A. M. Zanchettin, P. Rocco, A. Robertsson, and R. Johansson, “Exploiting task redundancy in industrial manipulators during drilling operations,” in *2011 IEEE International Conference on Robotics and Automation*. IEEE, 2011, pp. 128–133.
- [14] J. Baillieul, “Kinematic programming alternatives for redundant manipulators,” in *Proceedings. 1985 IEEE International Conference on Robotics and Automation*, vol. 2. IEEE, 1985, pp. 722–728.
- [15] B. Siciliano, “Kinematic control of redundant robot manipulators: A tutorial,” *Journal of intelligent and robotic systems*, vol. 3, no. 3, pp. 201–212, 1990.
- [16] A. Cimino, M. Elbasheer, F. Longo, G. Mirabelli, V. Solina, and P. Veltri, “Automatic simulation models generation in industrial systems: A systematic literature review and outlook towards simulation technology in the industry 5.0,” *Journal of Manufacturing Systems*, vol. 80, pp. 859–882, 2025.
- [17] Z. Ma, H. Guo, Y.-J. Gong, J. Zhang, and K. C. Tan, “Toward automated algorithm design: A survey and practical guide to meta-black-box-optimization,” *IEEE Transactions on Evolutionary Computation*, 2025.
- [18] J. Zhi, L.-F. Yu, and J.-M. Lien, “Designing human-robot coexistence space,” *IEEE Robotics and Automation Letters*, vol. 6, no. 4, pp. 7161–7168, 2021.
- [19] B. Ames, J. Morgan, and G. Konidaris, “Ikflow: Generating diverse inverse kinematics solutions,” *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 7177–7184, 2022.
- [20] P. Ruppel, N. Hendrich, S. Starke, and J. Zhang, “Cost functions to specify full-body motion and multi-goal manipulation tasks,” in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 3152–3159.
- [21] G. Du, Y. Zhang, X. Liu, R. J. Jiao, Y. Xia, and Y. Li, “A review of leader-follower joint optimization problems and mathematical models for product design and development,” *The International Journal of Advanced Manufacturing Technology*, vol. 103, pp. 3405–3424, 2019.
- [22] C. Cella, M. B. Robin, M. Faroni, A. M. Zanchettin, and P. Rocco, “Digital model-driven genetic algorithm for optimizing layout and task allocation in human-robot collaborative assemblies,” *arXiv preprint arXiv:2503.02774*, 2025.
- [23] P. I. Frazier, “A tutorial on bayesian optimization,” *arXiv preprint arXiv:1807.02811*, 2018.
- [24] D. Eriksson, M. Pearce, J. Gardner, R. D. Turner, and M. Poloczek, “Scalable global optimization via local bayesian optimization,” *Advances in neural information processing systems*, vol. 32, 2019.
- [25] E. Todorov, T. Erez, and Y. Tassa, “Mujoco: A physics engine for model-based control,” in *2012 IEEE/RSJ international conference on intelligent robots and systems*. IEEE, 2012, pp. 5026–5033.
- [26] N. Hansen, “The cma evolution strategy: a comparing review,” *Towards a new evolutionary computation: Advances in the estimation of distribution algorithms*, pp. 75–102, 2006.
- [27] J. J. Kuffner and S. M. LaValle, “Rrt-connect: An efficient approach to single-query path planning,” in *Proceedings 2000 ICRA. Millennium conference. IEEE international conference on robotics and automation. Symposia proceedings (Cat. No. 00CH37065)*, vol. 2. IEEE, 2000, pp. 995–1001.